

Developer Onboarding — Checklist

Tick each item as you finish. Push a PR every day, even if it's small.

Week 1 — Language, tooling, backend

Day 1 — Git + JS basics

- ☐ Installed git, configured name/email
- ☐ Cloned a repo, created a branch
- ☐ Made a commit with a clear message
- ☐ Opened a PR (README change is fine)
- ☐ Practiced: `let / const`, arrow functions, destructuring, spread
- ☐ Practiced: template literals, ES module `import / export`
- ☐ **PR pushed**

Day 2 — Async JS + TypeScript

- ☐ Wrote a function using Promises
- ☐ Rewrote it with `async / await`
- ☐ Added try/catch error handling
- ☐ Read through a `tsconfig.json` and understood each option
- ☐ Used `type` and `interface` — know when to pick which
- ☐ Wrote one generic function
- ☐ Rewrote a JS script in strict TS with zero `any`
- ☐ **PR pushed**

Day 3 — Node.js + project setup

- ☐ Initialized a project with `pnpm init`
- ☐ Added scripts to `package.json`
- ☐ Loaded env vars with `dotenv`
- ☐ Read/wrote files with `fs + path`
- ☐ Understood ESM vs CJS — know which one we use
- ☐ Built a CLI that reads CSV and writes JSON
- ☐ **PR pushed**

Day 4 — REST API

- ☐ Set up Express or Fastify with TS
- ☐ Implemented GET, POST, PATCH, DELETE on `/users`
- ☐ Added zod validation on request bodies
- ☐ Returned correct HTTP status codes
- ☐ Wrote a global error handler middleware
- ☐ Tested every endpoint with curl or Postman
- ☐ **PR pushed**

Day 5 — Postgres + SQL

- ☐ Installed Postgres locally
- ☐ Connected with `psql`
- ☐ Created a database, ran `CREATE TABLE`
- ☐ Wrote `SELECT` with `JOIN`
- ☐ Wrote a query inside a transaction
- ☐ Added an index, explained why
- ☐ Designed schema for contacts + messages
- ☐ Inserted seed data
- ☐ Wrote 5 sample queries
- ☐ **PR pushed**

Day 6 — Prisma

- ☐ Installed Prisma, ran `prisma init`
 - ☐ Wrote `schema.prisma` with relations
 - ☐ Ran `prisma migrate dev`
 - ☐ Ran `prisma generate`
 - ☐ Used `findMany`, `findUnique`, `create`, `update`, `delete`
 - ☐ Used `include` for a relation query
 - ☐ Wrote a transaction with `prisma.$transaction`
 - ☐ Spotted an N+1 query and fixed it
 - ☐ Rewrote Day 4 API to use Postgres + Prisma
 - ☐ **PR pushed**
-

Week 2 — Frontend + integration

Day 7 — React basics

- ☐ Scaffolded a Vite + React + TS app
- ☐ Built a component with props
- ☐ Used `useState` for local state
- ☐ Used `useEffect` for a side effect
- ☐ Rendered a list with proper `key` props
- ☐ Built a counter component
- ☐ Built a todo list (local state only)
- ☐ **PR pushed**

Day 8 — Data fetching + forms

- ☐ Installed React Query (or SWR)
- ☐ Fetched data from Day 6 API and rendered it
- ☐ Built a controlled form
- ☐ Added react-hook-form + zod resolver
- ☐ Implemented create, edit, delete flows
- ☐ Handled loading + error states
- ☐ **PR pushed**

Day 9 — Routing + UI

- ☐ Set up react-router
- ☐ Built a layout component (header/sidebar)
- ☐ Added `/contacts` list page
- ☐ Added `/contacts/:id` detail page
- ☐ Installed Tailwind
- ☐ Used shadcn/ui (or your team's library) for at least 3 components
- ☐ **PR pushed**

Day 10 — Auth

- ☐ Hashed passwords with `bcrypt`
- ☐ Built `/signup` and `/login` endpoints
- ☐ Issued a JWT (or session cookie)
- ☐ Wrote auth middleware on the backend
- ☐ Stored the token on the frontend
- ☐ Built a protected route component
- ☐ Configured CORS correctly

☐ PR pushed

Week 3 — Real-world skills

Day 11 — Queues + webhooks

- ☐ Installed Redis locally
- ☐ Set up BullMQ (or your queue)
- ☐ Created a producer that enqueues jobs
- ☐ Created a worker that processes jobs
- ☐ Added retries + backoff
- ☐ Built a webhook receiver endpoint
- ☐ Made the webhook idempotent
- ☐ PR pushed

Day 12 — Testing + debugging

- ☐ Wrote unit tests with vitest/jest
- ☐ Wrote integration tests for API routes with supertest
- ☐ Hit 80% coverage on Day 8's API
- ☐ Used Chrome DevTools to debug a React issue
- ☐ Used the Node debugger to step through backend code
- ☐ Read a stack trace and traced the root cause
- ☐ PR pushed

Day 13 — Observability + deploy

- ☐ Replaced `console.log` with structured logging (pino)
- ☐ Set up Sentry for error tracking
- ☐ Wrote a `Dockerfile`
- ☐ Wrote a `docker-compose.yml` for local dev
- ☐ Deployed the project to Railway/Render/Fly
- ☐ Watched logs in production
- ☐ PR pushed

Day 14 — Codebase orientation

- ☐ Cloned the real repos, got them running locally
- ☐ Walked through repo structure with mentor

- ☐ Understood naming conventions
- ☐ Read the CI/CD config
- ☐ Read the code review guide / norms
- ☐ Got access to Slack, Linear, GitHub, internal tools
- ☐ Picked one small bug or chore from the backlog

Day 15 — First real PR + domain

- ☐ Implemented the chore from Day 14
- ☐ Opened a PR through normal review process
- ☐ Addressed review comments
- ☐ Got the PR merged
- ☐ Sat through a 1-hour domain session: BSPs, WhatsApp templates, DLT, webhooks, opt-in/out
- ☐ Wrote a short doc summarizing what the platform does
- ☐ **PR merged in production repo**

this is **highlighter** text

Graduation criteria

You've completed onboarding when:

- ☐ You've shipped at least one merged PR to a real repo
 - ☐ You can build a full-stack CRUD feature (frontend + API + DB + auth) without help
 - ☐ You can debug a failing test or broken endpoint on your own
 - ☐ You know who to ask when you're stuck
-

heading one

heading two

heading three

fsjkgjs
this is bold

- vulle
 - rts
 - sdf
 - sd
- ☐ iymr omr

iteam					

header	table heading two	flgsfgkhfsh	hfkhgkjshhgsg
gmnskjfgksf	fkghskfhgkjfs	gdfkhgkjdfh	fdhgkhdfkgjdfh

-